

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



BIG DATA

Final Report

Building a Hybrid Search System for Academic Documents using Elasticsearch

Instructor(s): PGS.TS Thoai Nam
Student(s): Duong Gia An 2470293
Trinh Son Lam 2470287
Nguyen Thanh Huyen 2570416

HO CHI MINH CITY, MAY 2026

Contribution

Team Effort Breakdown:

Member	Name	Contribution	Effort (%)
1	Duong Gia An	System Architecture, Elasticsearch Indexing, Search Pipeline, Evaluation & Benchmarking	33.33%
2	Trinh Son Lam	Data Preprocessing, GPU Embedding Pipeline, Ground Truth Labeling	33.33%
3	Nguyen Thanh Huyen	Theoretical Background, Evaluation Analysis, Final Report Compilation	33.33%

Mục lục

Contribution

List of Figures	i
List of Tables	ii

1 Giới thiệu (Introduction)	1
1.1 Bối cảnh vấn đề	1
1.2 Mục tiêu dự án	2
1.3 Phạm vi (Scope)	2
2 Dataset (Dữ liệu)	4
2.1 Nguồn dữ liệu	4
2.2 Quy mô và Đặc điểm	4
2.3 Cấu trúc dữ liệu khai thác	4
2.4 Quy trình làm sạch dữ liệu (Data Cleaning)	5
3 Vấn đề và Giải pháp (Problems & Proposed Solutions)	6
3.1 Problem 1: Sự cứng nhắc của tìm kiếm từ khóa (The Rigidity of Key-word Search)	6
3.2 Problem 2: AI bỏ sót các thuật ngữ chuyên ngành (AI Misses Specialized Terminology)	6
3.3 Problem 3: Nhiều tìm kiếm đa ngành (Search Skewed by Multidisciplinary Noise)	7
4 Cơ sở lý thuyết (Theoretical Background)	8
4.1 Information Retrieval Pipeline	8
4.2 Inverted Index và Thuật toán BM25	8
4.3 Field Types trong Elasticsearch	9
4.4 Semantic Search và k-Nearest Neighbors (kNN)	10
4.5 Reciprocal Rank Fusion (RRF)	10
5 Kiến trúc hệ thống (System Architecture)	12
5.1 Sơ đồ luồng dữ liệu (Data Flow)	12
5.2 Luồng Tìm kiếm Hybrid Search (Query Pipeline)	12
5.3 Kiến trúc hạ tầng (Infrastructure)	13
5.3.1 Công nghệ sử dụng	13

5.3.2	Mô hình triển khai	14
5.3.3	Đặc tính Big Data	15
5.4	Index Mapping & Bulk Strategy	15
5.4.1	Mapping Definition	15
5.4.2	Bulk Indexing Strategy	16
5.5	Kiến trúc Dual-Index (Dual-Index Architecture)	16
5.5.1	Cơ chế Nạp bài báo mới (Incremental Ingestion)	16
6	Đường ống Xử lý Dữ liệu (Data Processing Pipeline)	18
6.1	Tổng quan Quy trình Xử lý Dữ liệu	18
6.2	Giai đoạn 1: Tiền xử lý Dữ liệu Quy mô lớn (Data Cleaning)	18
6.2.1	Kỹ thuật Đọc luồng (Streaming)	18
6.2.2	Các bước Làm sạch và Chuẩn hóa	19
6.3	Giai đoạn 2: Sinh Vector Nhúng (Embedding Generation)	19
6.3.1	Giải quyết Vấn đề Tràn Cửa sổ Ngữ cảnh	19
6.3.2	Tăng tốc bằng GPU đám mây (Google Colab)	20
7	Triển khai Cơ chế Tìm kiếm và Xếp hạng (Search Implementation)	22
7.1	Đánh Chỉ mục Tối ưu trên Elasticsearch (Indexing Optimization)	22
7.2	Truy vấn Tìm kiếm Từ khóa (BM25 Search)	23
7.3	Truy vấn Tìm kiếm Vector tương đồng (Semantic Search)	24
7.4	Truy vấn Kết hợp Hybrid Search dùng RRF	24
7.5	Kiến trúc Dual-Index cho Hybrid Search	26
8	Đánh giá Chất lượng Tìm kiếm (Evaluation & Analysis)	27
8.1	Phương pháp Xây dựng Tập Nhãn Kiểm định (Ground Truth Construction)	27
8.2	Các Chỉ số Đo lường Chất lượng Xếp hạng	28
8.2.1	Lý do Lựa chọn NDCG@10 và MRR@10	28
8.2.2	NDCG@10 (Normalized Discounted Cumulative Gain tại vị trí 10)	29
8.2.3	MRR@10 (Mean Reciprocal Rank tại vị trí 10)	29
8.3	Kết quả Thử nghiệm và Phân tích So sánh	30
8.3.1	Phân tích Nhóm A (Exact Keyword Queries)	30
8.3.2	Phân tích Nhóm B (Semantic Concept Queries)	31
8.3.3	Phân tích Nhóm C (Mixed/Hybrid Queries)	31
8.3.4	So sánh qua các Quy mô Dữ liệu (Scalability Quality Trends)	31
9	Đánh giá Hiệu năng Hệ thống (Performance Benchmarking)	33

9.1	Thiết lập Môi trường Thử nghiệm Hiệu năng	33
9.2	So sánh Thời gian Phản hồi (Query Latency Comparison)	34
9.2.1	Phân tích và Biện luận kết quả Latency	35
9.3	Đánh giá Dung lượng Lưu trữ (Index Storage Size)	35
9.4	Đánh đổi giữa Chất lượng Tìm kiếm và Chi phí Tài nguyên (Trade-off Analysis)	37
10	Giao diện Tìm kiếm (Search Interface)	38
10.1	Kiến trúc Client-Server	38
10.2	Các Endpoint API	38
10.3	Giao diện Người dùng (Frontend)	39
10.3.1	Thanh tìm kiếm và Chọn chế độ	39
10.3.2	Bảng lọc (Filters Panel)	39
10.3.3	Hiển thị kết quả	40
10.3.4	Thanh thống kê (Stats Bar)	40
10.4	Hướng dẫn Khởi chạy Hệ thống	40
11	Kết luận và Hướng phát triển (Conclusion & Future Work)	42
11.1	Kết quả Đạt được của Dự án	42
11.2	Ưu điểm và Hạn chế Hiện tại của Hệ thống	43
11.2.1	Ưu điểm	43
11.2.2	Hạn chế	43
11.3	Hướng phát triển trong Tương lai	44
	References	45

List of Figures

5.1.1 Sơ đồ luồng nạp dữ liệu (Data Ingestion Pipeline) từ dữ liệu thô đến Elasticsearch	12
5.2.1 Luồng tìm kiếm Hybrid Search: BM25 và Vector chạy song song, gộp kết quả bằng RRF	13
8.3.1 So sánh chất lượng NDCG@10	30
8.3.2 So sánh chất lượng MRR	30
8.3.3 Biểu đồ so sánh chất lượng tìm kiếm (NDCG@10 và MRR) giữa quy mô 100k và 1.2M	30
9.2.1 Độ trễ các phương thức ở quy mô 1.2M	34
9.2.2 Độ trễ tăng trưởng (P50) theo quy mô	34
9.2.3 Phân tích thời gian phản hồi hệ thống (Query Latency)	34
9.3.1 Tăng trưởng dung lượng Index trên Elasticsearch theo quy mô tài liệu	36

List of Tables

2.3.1 Cấu trúc dữ liệu được lập chỉ mục vào Elasticsearch	5
4.3.1 So sánh text và keyword field types trong Elasticsearch	9
8.3.1 Bảng so sánh chất lượng tìm kiếm NDCG@10 và MRR@10 trên các nhóm truy vấn ở quy mô 1.2M	30
9.2.1 Bảng so sánh thời gian phản hồi (Latency) theo phân vị P50 và P95 (đơn vị: mili-giây)	34
9.3.1 Dung lượng lưu trữ thực tế qua các quy mô dữ liệu	36
10.2.1 Các endpoint RESTful của FastAPI Backend	39

Chương 1

Giới thiệu (Introduction)

1.1 Bối cảnh vấn đề

Trong những năm gần đây, sự phát triển mạnh mẽ của các nghiên cứu khoa học đã tạo ra một sự bùng nổ về khối lượng tài liệu học thuật. Các kho lưu trữ mở như arXiv hiện đang chứa hàng triệu bài báo trải dài trên hàng trăm lĩnh vực khác nhau, tạo thành một nguồn dữ liệu khổng lồ (Big Data). Mặc dù sự phong phú này là một tài nguyên quý giá, nhưng nó cũng trực tiếp dẫn đến bài toán “quá tải thông tin”. Sinh viên và các nhà nghiên cứu đang phải dành một lượng thời gian khổng lồ chỉ để tra cứu, sàng lọc và đánh giá mức độ liên quan của tài liệu, thay vì tập trung vào việc nghiên cứu cốt lõi.

Đứng trước khối lượng dữ liệu khổng lồ này, các công cụ tìm kiếm học thuật truyền thống bắt đầu bộc lộ sự quá tải và những giới hạn về mặt kiến trúc.

- Nếu áp dụng các công cụ tìm kiếm thuần dựa trên từ khóa (Lexical Search), hệ thống sẽ vô cùng cứng nhắc, không thể hiểu được ngữ cảnh phức tạp, từ đồng nghĩa hay ý đồ thực sự của người tra cứu.
- Ngược lại, nếu chỉ lạm dụng các mô hình Trí tuệ nhân tạo (AI Search) để hiểu ngữ nghĩa, hệ thống lại dễ dàng đánh mất tính chính xác tuyệt đối—một yếu tố sống còn trong nghiên cứu khoa học khi người dùng cần tìm đích danh một tác giả cụ thể, một công thức, hay một thuật ngữ viết tắt chuyên ngành hiếm gặp. Hơn nữa, việc áp dụng các bộ lọc siêu dữ liệu (như năm xuất bản, danh mục) lên các hệ thống AI thường xuyên gây ra sự sai lệch hoặc làm giảm hiệu suất hệ thống đáng kể.

Với lượng dữ liệu lớn nhưng thiếu công cụ trích xuất thông minh đã đặt ra một yêu cầu cấp thiết: Cần phải có một kiến trúc tìm kiếm thể hệ mới, có khả năng xử lý Dữ liệu lớn với tốc độ cao, đồng thời dung hòa được cả độ chính xác của tìm kiếm từ khóa và sự thông minh của AI hiểu ngữ cảnh [6].

1.2 Mục tiêu dự án

Dự án hướng tới việc xây dựng một hệ thống tìm kiếm lai (Hybrid Search) toàn diện dành cho tài liệu học thuật, sử dụng Elasticsearch làm nền tảng cốt lõi. Bằng cách kết hợp ưu điểm của cả hai phương pháp: tìm kiếm văn bản truyền thống (Keyword Search) để đảm bảo độ chính xác tuyệt đối của các thuật ngữ chuyên ngành, và tìm kiếm vector (Vector Search) để nắm bắt ý nghĩa ngữ cảnh, hệ thống kỳ vọng sẽ khắc phục triệt để các hạn chế của các bộ máy tìm kiếm hiện tại, từ đó mang lại kết quả tra cứu chính xác và tối ưu nhất cho người dùng.

- **Về quản trị và xử lý dữ liệu:** Thiết lập quy trình thu thập, làm sạch và chuẩn hóa khối lượng lớn dữ liệu học thuật thô từ các nguồn mở. Đảm bảo dữ liệu được định dạng tối ưu, sẵn sàng cho việc lập chỉ mục (indexing) và truy xuất ở quy mô hàng triệu bản ghi.
- **Về kiến trúc nền tảng:** Triển khai một hạ tầng tìm kiếm phân tán mạnh mẽ, có khả năng lưu trữ và xử lý đồng thời cả dữ liệu văn bản thuần túy và các biểu diễn không gian vector đa chiều. Hệ thống phải đảm bảo tính sẵn sàng cao và phản hồi truy vấn với độ trễ thấp (low latency).
- **Về thuật toán truy xuất cốt lõi:** Phát triển cơ chế tìm kiếm đa luồng, cho phép thực thi song song các truy vấn dựa trên từ vựng và truy vấn dựa trên ngữ nghĩa. Trọng tâm là xây dựng cấu trúc tự động đánh giá, tính điểm và hợp nhất xếp hạng từ các nguồn kết quả khác nhau để tối ưu hóa độ chính xác.
- **Về giao diện tương tác:** Xây dựng giao diện web hiện đại kết nối với backend API, hỗ trợ chuyển đổi giữa các chế độ tìm kiếm (BM25, Vector, Hybrid) và lọc kết quả theo năm, danh mục trực quan.
- **Về tính ứng dụng và đánh giá:** Hoàn thiện giao diện tương tác trực quan, hỗ trợ người dùng thực hiện các thao tác tra cứu và lọc dữ liệu phức tạp. Đồng thời, áp dụng các bộ tiêu chí đánh giá định lượng khoa học để chứng minh hiệu năng của mô hình Tìm kiếm Lai so với các phương pháp truyền thống.

1.3 Phạm vi (Scope)

- **Về nguồn tài nguyên dữ liệu:** Dự án khai thác kho lưu trữ tài liệu học thuật mở arXiv thông qua bộ dữ liệu từ Kaggle. Đây là nguồn tài nguyên tri thức đa ngành quy mô lớn, cung cấp hệ thống thuộc tính phong phú từ nội dung văn bản cho đến các siêu dữ liệu phân loại.



- **Về quy mô dữ liệu:** Hệ thống được triển khai trên toàn bộ 1,203,108 bài báo thuộc lĩnh vực Khoa học Máy tính, mỗi bài báo kèm vector nhúng 384 chiều. Đồng thời, các thử nghiệm đánh giá được thực hiện trên nhiều quy mô (100k, 500k, 1.2M) để kiểm chứng khả năng mở rộng.
- **Về hạ tầng kỹ thuật:** Hệ thống Elasticsearch 8.12 cục bộ trên Docker với kiến trúc Dual-Index (`arxiv_text` cho BM25, `arxiv_vectors` cho kNN) phục vụ toàn bộ 1.2M bài báo. Backend FastAPI và Frontend React cung cấp giao diện tương tác đầy đủ.
- **Về phương pháp luận:** Trọng tâm là hoàn thiện hệ thống Hybrid Search sử dụng thuật toán Reciprocal Rank Fusion (RRF) kết hợp tìm kiếm BM25 và vector search, cùng bộ đánh giá chất lượng NDCG@10 và MRR@10.

Chương 2

Dataset (Dữ liệu)

2.1 Nguồn dữ liệu

Dự án sử dụng bộ dữ liệu arXiv từ Kaggle [2] (<https://www.kaggle.com/datasets/Cornell-University/arxiv>), một kho lưu trữ mở chứa các bài báo học thuật do Cornell University duy trì và cập nhật liên tục.

2.2 Quy mô và Đặc điểm

- **Tổng số lượng:** Hơn 2.7 triệu bài báo. Đây là một tập dữ liệu có tính tích lũy cao, được cập nhật liên tục.
- **Dung lượng file gốc:** Hơn 3GB (file JSON dạng line-delimited, mỗi dòng là một bản ghi JSON).
- **Độ phủ:** Hơn 130 lĩnh vực (Computer Science, Physics, Mathematics,...).
- **Tập con sử dụng:** Toàn bộ 1,203,108 bài báo thuộc lĩnh vực Computer Science (`cs.*`).

2.3 Cấu trúc dữ liệu khai thác

Bảng 2.3.1 mô tả cấu trúc các trường dữ liệu được trích xuất và sử dụng trong hệ thống:

Trường	Kiểu gốc	ES Type	Mô tả & Vai trò
id	string	keyword	Mã định danh duy nhất của bài báo trên arXiv
title	string	text	Tiêu đề bài báo – phục vụ tìm kiếm BM25 (boost $\times 2$)
abstract	string	text	Tóm tắt nội dung – phục vụ tìm kiếm BM25
categories	string	keyword	Danh mục phân loại (VD: <code>cs.AI</code> , <code>cs.LG</code>) – phục vụ exact-match filtering
authors	string	text	Danh sách tác giả
year	string	keyword	Năm cập nhật – phục vụ exact-match filtering

Table 2.3.1: Cấu trúc dữ liệu được lập chỉ mục vào *Elasticsearch*

2.4 Quy trình làm sạch dữ liệu (Data Cleaning)

Do file gốc có dung lượng hơn 3GB, việc đọc toàn bộ vào bộ nhớ (RAM) cùng lúc là không khả thi trên hầu hết cấu hình máy tính phổ thông. Nhóm đã áp dụng kỹ thuật **streaming (đọc từng dòng)** để xử lý tuần tự, giúp kiểm soát tốt bộ nhớ ngay cả khi kích thước dữ liệu lên đến hàng GB. Quy trình cụ thể bao gồm:

1. **Đọc tuần tự (Line-by-line streaming):** Mở file gốc và xử lý từng dòng JSON, tránh tải toàn bộ file vào RAM.
2. **Lọc danh mục (Category filtering):** Chỉ giữ lại các bản ghi có trường `categories` chứa tiền tố `cs.` (Computer Science).
3. **Loại bỏ bản ghi lỗi:** Bỏ qua các dòng không parse được (malformed JSON) và các bản ghi thiếu trường `title` hoặc `abstract`.
4. **Chuẩn hóa văn bản:** Loại bỏ ký tự xuống dòng (`\n`) trong `title` và `abstract`, trim khoảng trắng dư thừa.
5. **Trích xuất metadata:** Tách trường `year` từ `update_date` (định dạng YYYY-MM-DD).
6. **Xuất file:** Lưu kết quả dưới dạng JSONL chuẩn để phục vụ cho bước Sinh Vector Nhúng (Embedding) và Bulk Indexing tiếp theo.

Chương 3

Vấn đề và Giải pháp (Problems & Proposed Solutions)

3.1 Problem 1: Sự cứng nhắc của tìm kiếm từ khóa (The Rigidity of Keyword Search)

Vấn đề: Các hệ thống tìm kiếm dựa trên từ khóa truyền thống hoạt động theo cơ chế đối khớp chuỗi ký tự (string matching). Người dùng phải gõ chính xác từ khóa xuất hiện trong tài liệu. Các từ đồng nghĩa (synonyms), cách diễn đạt khác (rephrasing), hoặc biến thể ngữ pháp đều không được nhận diện. Ví dụ: truy vấn “*methods to detect fake images*” sẽ không trả về các bài báo chứa “*deepfake detection*” hoặc “*generative adversarial network forensics*”, dù chúng có nội dung liên quan trực tiếp.

Hướng giải pháp: Tích hợp **Vector Search** bên cạnh BM25 truyền thống. Ý tưởng cốt lõi là chuyển đổi toàn bộ tài liệu và câu truy vấn thành các vector số thực trong không gian ngữ nghĩa đa chiều bằng mô hình học sâu (Sentence Transformer). Khi đó, việc tìm kiếm trở thành bài toán tìm các vector gần nhất (nearest neighbors) trong không gian này — cho phép hệ thống hiểu được *ý nghĩa ngữ cảnh* thay vì chỉ khớp từng từ. Giải thuật HNSW [5] đảm bảo tốc độ tìm kiếm $O(\log N)$ ngay cả trên tập dữ liệu hàng triệu tài liệu.

3.2 Problem 2: AI bỏ sót các thuật ngữ chuyên ngành (AI Misses Specialized Terminology)

Vấn đề: Các mô hình AI hiểu ngữ cảnh tốt nhưng lại kém chính xác khi người dùng cần tìm kiếm đích danh một thuật ngữ viết tắt hiếm gặp (ví dụ: “*ViT-B/16*”), tên tác giả cụ thể (“*Vaswani et al.*”), hoặc ký hiệu toán học. Mô hình nhúng có xu hướng “mờ hóa” (blur) các thuật ngữ ngắn và chuyên biệt vào không gian ngữ nghĩa chung, dẫn đến mất tính chính xác tuyệt đối — một yếu tố sống còn trong nghiên cứu khoa học.

Hướng giải pháp: Thay vì chỉ dựa vào một phương pháp, hệ thống chạy **song song cả hai luồng** tìm kiếm và gộp kết quả bằng thuật toán **Reciprocal Rank Fusion (RRF)** [1]:

- BM25 đảm nhận đối khớp chính xác từ khóa qua Inverted Index — đảm bảo không bỏ sót các thuật ngữ viết tắt và tên riêng.
- Vector search đảm nhận đối khớp ngữ nghĩa qua Cosine Similarity — bổ sung các tài liệu liên quan mà BM25 bỏ lỡ.
- RRF gộp hai danh sách kết quả dựa trên *thứ hạng* (không phải điểm số, vốn có thang đo khác nhau giữa hai phương pháp), đẩy các tài liệu được cả hai phương pháp đồng thuận lên đầu kết quả.

3.3 Problem 3: Nhiều tìm kiếm đa ngành (Search Skewed by Multidisciplinary Noise)

Vấn đề: Tập dữ liệu arXiv chứa hơn 1.2 triệu bài báo thuộc nhiều lĩnh vực phụ của Khoa học Máy tính (cs.AI, cs.CV, cs.CL, cs.SE, ...). Một từ khóa như “*transformer*” có thể xuất hiện trong hàng chục nghìn bài báo thuộc nhiều lĩnh vực khác nhau (NLP, Computer Vision, Robotics, ...). Nếu không có bộ lọc siêu dữ liệu (metadata filters), người dùng bị ngập trong kết quả nhiều không thuộc lĩnh vực quan tâm. Hơn nữa, việc áp dụng bộ lọc cứng (năm, danh mục) lên hệ thống AI vector thường gây ra xung đột kiến trúc: bộ lọc phải được thực thi *trước* hoặc *sau* tìm kiếm vector, dẫn đến mất kết quả hoặc tăng độ trễ.

Hướng giải pháp: Sử dụng Elasticsearch với cơ chế **lọc tích hợp (Integrated Filtering)** ở cấp truy vấn DSL. Các trường siêu dữ liệu (năm, danh mục) được đánh chỉ mục dưới dạng **keyword**, cho phép exact-match filtering diễn ra *đồng thời* với tìm kiếm BM25 và Vector thay vì hậu xử lý. Elasticsearch tự động chuyển đổi giữa các thuật toán quét (scanning algorithms) dựa trên độ nghiêm ngặt (selectivity) của bộ lọc, đảm bảo phản hồi nhanh và chính xác mà không cần can thiệp thủ công. Giao diện tìm kiếm cho phép người dùng thu hẹp phạm vi tìm kiếm theo khoảng năm và danh mục trực tiếp trên thanh công cụ.

Chương 4

Cơ sở lý thuyết (Theoretical Background)

4.1 Information Retrieval Pipeline

Với hệ thống truy xuất thông tin (Information Retrieval - IR), thay vì thực hiện thao tác rà soát tuần tự trên toàn bộ cơ sở dữ liệu vốn không khả thi với dữ liệu lớn, hệ thống IR hoạt động dựa trên một quy trình xử lý luồng (pipeline) khép kín. Quy trình này chịu trách nhiệm chuyển đổi khối lượng lớn dữ liệu phi cấu trúc thành các ma trận toán học và cấu trúc lưu trữ tối ưu, bao gồm ba giai đoạn chính:

1. **Ingestion (Thu thập và làm sạch):** Dữ liệu thô từ nhiều nguồn được thu thập, tiến hành chuẩn hóa định dạng và loại bỏ các thành phần nhiễu giúp bóc tách và trích xuất các trường thông tin quan trọng nhằm tạo ra một tập dữ liệu đầu vào sạch, đáp ứng yêu cầu dữ liệu đầu vào cho các bước xử lý tiếp theo.
2. **Indexing (Lập chỉ mục):** Dữ liệu sau khi làm sạch được tổ chức thành các cấu trúc dữ liệu đặc biệt (như Inverted Index đối với văn bản) để tối ưu hóa tốc độ tìm kiếm.
3. **Query Processing (Xử lý truy vấn):** Hệ thống tiếp nhận câu truy vấn từ người dùng, phân tích (tokenize), và đối chiếu với chỉ mục để trả về các kết quả phù hợp nhất dựa trên thuật toán xếp hạng (ranking algorithm).

4.2 Inverted Index và Thuật toán BM25

Trong giai đoạn hiện tại, nhằm đáp ứng yêu cầu tìm kiếm với hiệu năng cao trên các tập dữ liệu lớn, hệ thống sử dụng các công nghệ tìm kiếm văn bản cốt lõi:

- **Inverted Index (Chỉ mục đảo ngược):** Đây là cấu trúc dữ liệu nền tảng của Elasticsearch. Thay vì quét từng tài liệu để tìm từ khóa, Inverted Index lưu trữ một danh sách các từ khóa (terms) và ánh xạ mỗi từ khóa đến danh

sách các tài liệu (documents) chứa nó. Điều này giúp tốc độ tìm kiếm đạt mức gần thời gian thực (near real-time) ngay cả với Big Data.

- **BM25 (Best Matching 25):** Là thuật toán xếp hạng mặc định của Elasticsearch, được phát triển và cải tiến dựa trên nền tảng của mô hình trọng số TF-IDF (Term Frequency - Inverse Document Frequency) [8]. Thuật toán này giúp khai phá văn bản và truy xuất thông tin, đánh giá và xếp hạng độ liên quan của các tài liệu trả về so với câu truy vấn của người dùng dựa trên:
 - **TF (Term Frequency):** Tần suất xuất hiện của từ khóa trong tài liệu. Tần suất càng cao, điểm số càng tăng, nhưng có giới hạn bão hòa để tránh việc lặp từ khóa quá mức.
 - **IDF (Inverse Document Frequency):** Độ hiếm của từ khóa trên toàn bộ tập dữ liệu. Các từ khóa xuất hiện trong ít tài liệu (như thuật ngữ chuyên ngành) sẽ có trọng số cao hơn so với các từ thông dụng (như "the", "a", "is").

4.3 Field Types trong Elasticsearch

Elasticsearch phân biệt rõ ràng hai loại dữ liệu văn bản phục vụ cho các mục đích truy vấn khác nhau [4]. Bảng 4.3.1 tóm tắt sự khác biệt:

Đặc điểm	text	keyword
Phân tích cú pháp	Có (tokenization, lowercasing, stop-word removal)	Không – lưu nguyên bản
Cấu trúc lưu trữ	Inverted Index (ánh xạ term → documents)	Doc Values (ánh xạ document → giá trị gốc)
Loại truy vấn	Full-text search (BM25 scoring)	Exact-match, filtering, aggregation
Ví dụ trường	title, abstract, authors	year, categories, id
Ví dụ truy vấn	“machine learning” → khớp “Machine Learning for...”	“2023” → chỉ khớp chính xác “2023”

Table 4.3.1: So sánh *text* và *keyword* field types trong Elasticsearch

Việc lựa chọn đúng field type cho từng trường dữ liệu là yếu tố then chốt ảnh hưởng trực tiếp đến hiệu năng tìm kiếm và tính chính xác của kết quả trả về.

4.4 Semantic Search và k-Nearest Neighbors (kNN)

Để giải quyết hạn chế của tìm kiếm từ khóa truyền thống (chưa nắm bắt được quan hệ đồng nghĩa, ngữ cảnh ngữ nghĩa), hệ thống áp dụng các mô hình ngôn ngữ học sâu (như **SentenceTransformers** [7]) để chuyển đổi toàn bộ thông tin văn bản của bài báo thành các vector số thực mật độ cao (dense vectors). Khoảng cách không gian giữa các vector (ví dụ: độ tương đồng Cosine Similarity) phản ánh mức độ tương đồng về mặt khái niệm giữa các tài liệu.

Tuy nhiên, việc tính toán khoảng cách k-Nearest Neighbors (kNN) chính xác trên tập dữ liệu lớn đòi hỏi phép quét tuyến tính $O(N)$ so sánh vector truy vấn với toàn bộ vector trong cơ sở dữ liệu. Điều này hoàn toàn bất khả thi đối với Big Data do độ trễ truy vấn quá cao. Để giải quyết rào cản này, Elasticsearch tích hợp giải thuật tìm kiếm láng giềng gần đúng **HNSW** (Hierarchical Navigable Small World) [5]:

- **Cấu trúc Đồ thị đa tầng (Hierarchical Graph):** HNSW xây dựng cấu trúc đồ thị đa lớp tương tự như danh sách liên kết nhảy (Skip Lists). Lớp trên cùng thưa thớt giúp định vị nhanh khu vực lân cận của vector đích, các lớp dưới dày đặc dần giúp thu hẹp phạm vi và tìm kiếm chính xác ở khoảng cách gần.
- **Độ phức tạp tính toán:** Giải thuật chuyển độ phức tạp tìm kiếm kNN từ tuyến tính $O(N)$ về mức logarithm $O(\log N)$, cho phép tìm kiếm ngữ nghĩa trên hàng triệu tài liệu trong thời gian mili-giây.
- **Cân bằng Trade-off:** Người dùng có thể cân bằng giữa tốc độ tìm kiếm và độ chính xác thu hồi (Recall) thông qua tham số `num_candidates` khi truy vấn.

4.5 Reciprocal Rank Fusion (RRF)

RRF là một thuật toán xếp hạng (ranking algorithm) kết hợp kết quả từ nhiều phương pháp tìm kiếm khác nhau (ở đây là BM25 và kNN) mà không cần phải chuẩn hóa điểm số (score) của chúng [1]. Công thức RRF tính điểm cho mỗi document d dựa trên vị trí xếp hạng trong từng hệ thống:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

trong đó R là tập các bảng xếp hạng (từ BM25 và kNN), $r(d)$ là vị trí của document



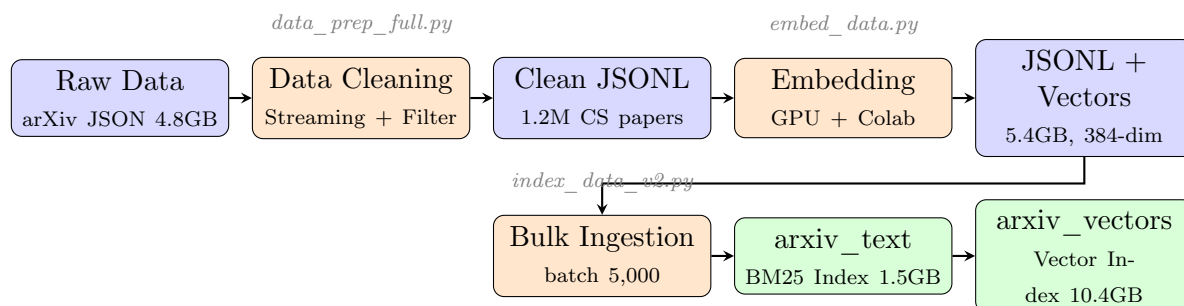
d trong bảng xếp hạng r , và k là hằng số điều chỉnh (thường $k = 60$). Công thức này giúp đẩy các tài liệu xuất hiện ở thứ hạng cao trong cả hai phương pháp lên vị trí top đầu, mang lại kết quả tối ưu và chính xác nhất cho hệ thống Hybrid Search.

Chương 5

Kiến trúc hệ thống (System Architecture)

5.1 Sơ đồ luồng dữ liệu (Data Flow)

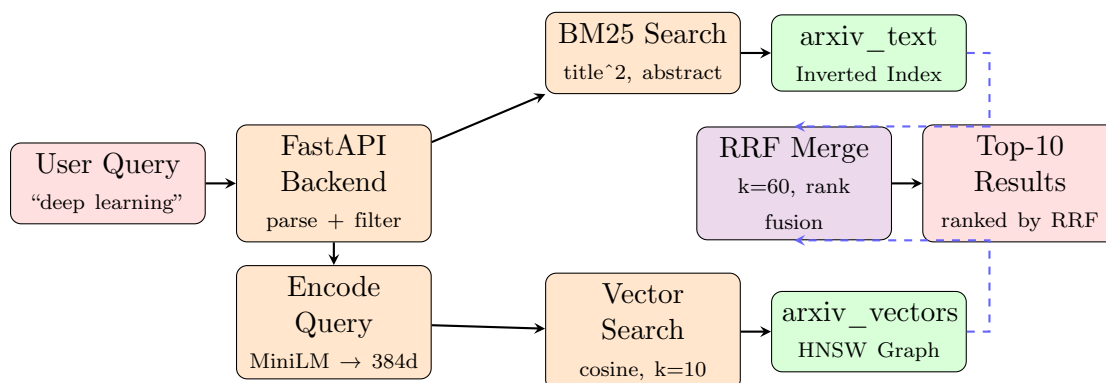
Quy trình xử lý dữ liệu của hệ thống được thiết kế theo dạng đường ống (pipeline) tuyến tính, bao gồm hai luồng chính: luồng nạp dữ liệu (Data Ingestion) và luồng truy vấn (Query Pipeline), được minh họa trong Hình 5.1.1:



Hình 5.1.1: Sơ đồ luồng nạp dữ liệu (Data Ingestion Pipeline) từ dữ liệu thô đến *Elasticsearch*

5.2 Luồng Tìm kiếm Hybrid Search (Query Pipeline)

Khi người dùng nhập truy vấn, hệ thống thực hiện tìm kiếm kết hợp (Hybrid Search) theo luồng được minh họa trong Hình 5.2.1:



Hình 5.2.1: *Luồng tìm kiếm Hybrid Search: BM25 và Vector chạy song song, gộp kết quả bằng RRF*

Luồng truy vấn gồm các bước:

1. Người dùng nhập truy vấn qua giao diện web. FastAPI Backend tiếp nhận, phân tích bộ lọc (năm, danh mục).
2. **Luồng BM25:** Gửi truy vấn đến chỉ mục `arxiv_text`, Elasticsearch phân tích cú pháp (tokenize, lowercase) và tính điểm BM25 dựa trên Inverted Index.
3. **Luồng Vector search:** Mô hình `all-MiniLM-L6-v2` chuyển câu truy vấn thành vector 384 chiều. Vector này được gửi đến chỉ mục `arxiv_vectors` để tìm k láng giềng gần nhất trên đồ thị HNSW.
4. **RRF Merge:** Hai danh sách kết quả được gộp bằng công thức RRF (xem Chương 7). Các tài liệu xuất hiện ở thứ hạng cao trong cả hai luồng được đẩy lên đầu.
5. Trả về top-10 kết quả cuối cùng cho giao diện người dùng kèm điểm RRF và metadata.

5.3 Kiến trúc hạ tầng (Infrastructure)

5.3.1 Công nghệ sử dụng

- **Python 3.x:** Xử lý dữ liệu (Data Processing) và tương tác với Elasticsearch thông qua thư viện `elasticsearch-py`.

- **Elasticsearch 8.12:** Search Engine cốt lõi, hỗ trợ Inverted Index, BM25 scoring, và filtering [4].
- **Docker & Docker Compose:** Đóng gói và triển khai toàn bộ hạ tầng (Elasticsearch + Kibana) trong các container độc lập [3].
- **FastAPI:** Backend API cung cấp các endpoint RESTful cho giao diện tìm kiếm (xem Chương 10).
- **React + Vite:** Giao diện người dùng tương tác, kết nối FastAPI qua HTTP.
- **Kibana 8.12:** Giao diện quản trị và trực quan hóa dữ liệu, hỗ trợ Dev Tools để thử nghiệm truy vấn.

5.3.2 Mô hình triển khai

Hệ thống được triển khai dưới dạng Single-node cluster trên môi trường Docker cục bộ, sử dụng cấu hình Docker Compose như sau:

```
1 services:
2   elasticsearch:
3     image: elasticsearch:8.12.2
4     environment:
5       - discovery.type=single-node
6       - xpack.security.enabled=false
7       - "ES_JAVA_OPTS=-Xms2g -Xmx2g"
8     ports:
9       - "9200:9200"
10    volumes:
11      - es_data:/usr/share/elasticsearch/data
12
13   kibana:
14     image: kibana:8.12.2
15     environment:
16       - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
17     ports:
18       - "5601:5601"
```

Program 5.1: *Docker Compose – Elasticsearch & Kibana*

5.3.3 Đặc tính Big Data

Mặc dù chạy trên single-node, kiến trúc bên trong của Elasticsearch vẫn duy trì các khái niệm phân tán như **Index**, **Shard** (phân mảnh dữ liệu) và **Replica** (bản sao). Điều này cho phép hệ thống dễ dàng mở rộng (scale-out) bằng cách thêm các node mới vào cluster khi khối lượng dữ liệu tăng lên mức hàng triệu bài báo mà không cần thay đổi kiến trúc ứng dụng.

5.4 Index Mapping & Bulk Strategy

5.4.1 Mapping Definition

Lược đồ dữ liệu (Mapping) được định nghĩa tường minh để tối ưu hóa hiệu năng cho từng loại truy vấn:

```
1 {  
2   "mappings": {  
3     "properties": {  
4       "id":          { "type": "keyword" },  
5       "title":       { "type": "text" },  
6       "abstract":    { "type": "text" },  
7       "categories":  { "type": "keyword" },  
8       "authors":     { "type": "text" },  
9       "year":        { "type": "keyword" }  
10    }  
11  },  
12  "settings": {  
13    "number_of_shards": 1,  
14    "number_of_replicas": 0  
15  }  
16 }
```

Program 5.2: *Elasticsearch Index Mapping*

Trong đó:

- Các trường `title`, `abstract`, `authors` được map dưới dạng `text` – Elasticsearch tự động thực hiện phân tích cú pháp (tokenization, lowercasing, stop-word removal) và xây dựng Inverted Index để phục vụ tìm kiếm BM25.
- Các trường `id`, `year`, `categories` được map dưới dạng `keyword` – lưu trữ

nguyên bản, không qua phân tích, phục vụ exact-match filtering với tốc độ $O(1)$.

5.4.2 Bulk Indexing Strategy

Thay vì gửi từng tài liệu qua HTTP POST riêng lẻ (rất chậm và tốn tài nguyên mạng), hệ thống sử dụng hàm `helpers.bulk()` của thư viện `elasticsearch-py`. Dữ liệu được gom thành các lô (batch) với kích thước **5,000 documents** mỗi lần gửi, giúp:

- Giảm số lượng HTTP round-trip từ hàng triệu xuống còn vài trăm requests.
- Tối ưu hóa băng thông mạng và giảm tải cho bộ nhớ heap của Elasticsearch.
- Toàn bộ 1,203,108 documents được index trong thời gian hợp lý.

5.5 Kiến trúc Dual-Index (Dual-Index Architecture)

Để tối ưu hóa hiệu năng tìm kiếm và hỗ trợ nạp bài báo mới (incremental ingestion) mà không cần đánh chỉ mục lại toàn bộ, hệ thống được nâng cấp lên kiến trúc Dual-Index trên cùng một cụm Elasticsearch:

- **Index `arxiv_text`:** Chỉ chứa các trường văn bản (`title`, `abstract`, `authors`, `year`, `categories`), phục vụ tìm kiếm BM25. Kích thước chỉ 1.5GB cho 1.2 triệu bài báo do không lưu trữ vector nhúng.
- **Index `arxiv_vectors`:** Chứa toàn bộ metadata và vector nhúng 384 chiều (`embedding`), phục vụ tìm kiếm vector. Kích thước 10.4GB.

5.5.1 Cơ chế Nạp bài báo mới (Incremental Ingestion)

Khi có bài báo mới, hệ thống tự động:

1. Sinh vector nhúng 384 chiều bằng mô hình `all-MiniLM-L6-v2`.
2. Đẩy metadata vào `arxiv_text` qua Elasticsearch Index API.
3. Đẩy ID và vector vào `arxiv_vectors` qua Elasticsearch Index API.
4. Refresh cả hai index để bài báo mới có thể tìm kiếm được ngay lập tức.



Cơ chế này cho phép bổ sung bài báo mới trong thời gian thực mà không cần nạp lại toàn bộ 1.2 triệu bài báo từ đầu.

Chương 6

Đường ống Xử lý Dữ liệu (Data Processing Pipeline)

6.1 Tổng quan Quy trình Xử lý Dữ liệu

Để chuyển đổi tập dữ liệu thô từ arXiv (4.8GB JSON) thành tập dữ liệu sẵn sàng cho tìm kiếm, hệ thống thực hiện một quy trình xử lý gồm ba giai đoạn chính, được thiết kế theo mô hình đường ống (pipeline) để đảm bảo tính module hóa và khả năng xử lý Big Data:

1. **Giai đoạn 1 — Tiền xử lý và Làm sạch (Data Cleaning):** Đọc, lọc và chuẩn hóa dữ liệu thô bằng kỹ thuật streaming.
2. **Giai đoạn 2 — Sinh Vector Nhúng (Embedding Generation):** Chuyển đổi văn bản thành vector số thực 384 chiều bằng mô hình học sâu.
3. **Giai đoạn 3 — Đánh Chỉ mục (Indexing):** Nạp dữ liệu sạch và vector nhúng vào Elasticsearch qua Bulk API.

6.2 Giai đoạn 1: Tiền xử lý Dữ liệu Quy mô lớn (Data Cleaning)

6.2.1 Kỹ thuật Đọc luồng (Streaming)

File dữ liệu gốc từ Kaggle có dung lượng rất lớn (~4.8GB dạng JSON). Nếu đọc toàn bộ file vào RAM bằng hàm `json.load()`, hệ thống sẽ lập tức cạn kiệt bộ nhớ và bị ngắt kết nối (OOM — Out of Memory). Script tiền xử lý `data_prep_full.py` khắc phục điều này bằng kỹ thuật **đọc luồng tuyến tính (Line-by-line Streaming)**: đọc tệp tin theo từng dòng đọc lập (mỗi dòng chứa thông tin của một bài báo dưới dạng JSON), phân tích và xử lý tuần tự trước khi ghi trực tiếp dòng kết quả xuống đĩa cứng theo định dạng JSONL (JSON Lines). Kỹ thuật này cho phép xử lý file có dung lượng *lớn hơn* RAM vật lý của máy.

6.2.2 Các bước Làm sạch và Chuẩn hóa

- **Trích xuất bài báo Computer Science:** Lọc các bài báo dựa trên trường danh mục (`categories`). Chỉ các bài báo chứa nhãn `cs.*` mới được chọn. Qua bộ lọc này, số lượng bài báo giảm từ hơn 2.4 triệu xuống còn 1,203,108 bài.
- **Sửa lỗi trường năm xuất bản (Year Parsing):** Thay vì sử dụng trường ngày cập nhật hệ thống (`update_date`) vốn phản ánh lần sửa đổi cuối cùng của bài báo, hệ thống trích xuất năm trực tiếp từ mã định danh arXiv (`id`) theo quy luật:
 - Dạng định danh mới (`YYMM.nnnnn`): ví dụ 2301.00001 trích xuất 2 ký tự đầu làm năm (2023).
 - Dạng định danh cũ (`cs/YYMMnnn`): nếu không có dấu chấm phân tách, hệ thống sử dụng thuật toán fallback quay lại đọc năm từ trường ngày cập nhật hệ thống.
- **Chuẩn hóa văn bản:** Loại bỏ ký tự xuống dòng (`\n`) trong `title` và `abstract`, thay thế bằng khoảng trắng đơn để đảm bảo tính nhất quán khi lập chỉ mục.

6.3 Giai đoạn 2: Sinh Vector Nhúng (Embedding Generation)

6.3.1 Giải quyết Vấn đề Tràn Cửa sổ Ngữ cảnh

Mô hình nhúng văn bản được chọn là `all-MiniLM-L6-v2` có giới hạn chiều dài đầu vào tối đa là **256 tokens** (mỗi token tiếng Anh trung bình khoảng 4 ký tự). Văn bản đầu vào gửi đến mô hình được ghép nối theo cấu trúc:

$$T_{\text{input}} = \text{Title} \parallel [\text{SEP}] \parallel \text{Abstract} \quad (6.1)$$

Trong đó tiêu đề (Title) trung bình chiếm khoảng 15 tokens và ký tự phân tách đặc biệt `[SEP]` chiếm 1 token. Do đó, ngân sách tối đa dành cho tóm tắt (Abstract) chỉ còn khoảng **240 tokens**, tương đương khoảng **960 ký tự**.

Nếu đưa một Abstract quá dài (nhiều Abstract của arXiv dài từ 1500 – 3000 ký tự) trực tiếp vào hàm `model.encode()`, mô hình sẽ tự động thực hiện cắt cứng (hard cut) tại token thứ 256, có thể gây mất thông tin ngữ nghĩa quan trọng ở cuối Abstract.

Để giải quyết vấn đề này, chúng tôi thiết kế thuật toán **Cắt Abstract Thông minh (Smart Abstract Truncation)** trước khi thực hiện nhúng:

Algorithm 1: Thuật toán Cắt Abstract Thông minh theo ranh giới câu

Input: Abstract text A , Maximum characters limit $M = 960$

Output: Truncated Abstract A_{trunc}

if *length of* $A \leq M$ **then**

return A

$C \leftarrow$ first M characters of A

$I_{period} \leftarrow$ index of last period "." in C

if $I_{period} > M \times 0.6$ **then**

return $C[0 \dots I_{period}]$

else

return C

Thuật toán này đảm bảo văn bản Abstract được cắt tại ranh giới của một câu hoàn chỉnh nếu khoảng cách cắt chấp nhận được (lớn hơn 60% chiều dài giới hạn), giúp duy trì cấu trúc câu tự nhiên và chất lượng ngữ nghĩa của vector biểu diễn tốt nhất.

6.3.2 Tăng tốc bằng GPU đám mây (Google Colab)

Khi nâng cấp hệ thống lên quy mô 1.2 triệu bài báo, việc sinh vector nhúng trên CPU cục bộ gặp phải rào cản lớn: tốc độ chỉ đạt 5 – 10 tài liệu/giây, tương đương 33 – 66 giờ chạy liên tục. Do đó, chúng tôi chuyển sang môi trường đám mây **Google Colab** sử dụng GPU NVIDIA Tesla T4 (16GB VRAM):

- **Cấu hình lô (Batching):** Sử dụng kích thước lô tối ưu là **512 tài liệu** mỗi lượt suy luận, tận dụng tối đa băng thông bộ nhớ GPU mà không gây lỗi tràn VRAM.
- **Hiệu năng thực tế:** Tốc độ sinh vector đạt trung bình **500 – 800 tài liệu/giây** (nhanh gấp ~ 80 lần so với CPU). Toàn bộ 1,203,108 bài báo hoàn thành trong vòng **35 phút**.
- **Cơ chế Checkpoint và Resume:** Hệ thống ghi trực tiếp kết quả xuống Google Drive theo định dạng JSONL. Cứ mỗi **50.000 tài liệu**, chương trình tự động ghi file trạng thái `progress.json`. Nếu phiên Colab bị ngắt kết nối, hệ thống tự động đọc tệp trạng thái và tiếp tục sinh vector từ chỉ mục hiện tại mà không phải tính toán lại từ đầu.



Kết quả của giai đoạn này là tệp tin `arxiv_cs_full_with_vectors.jsonl` có dung lượng khoảng **5.4 GB**, sẵn sàng cho quá trình đánh chỉ mục tại Elasticsearch cục bộ.

Chương 7

Triển khai Cơ chế Tìm kiếm và Xếp hạng (Search Implementation)

7.1 Đánh Chỉ mục Tối ưu trên Elasticsearch (Indexing Optimization)

Để chuẩn bị môi trường cho việc tìm kiếm kết hợp Hybrid Search, chúng tôi tối ưu cấu hình chỉ mục (Index Settings) trên Elasticsearch phiên bản V2 (`arxiv_papers_v2`) nhằm thích ứng với tài nguyên giới hạn của hệ thống single-node (16GB RAM):

- **Cấu hình Phân mảnh (Shard Settings):** Giảm số lượng phân mảnh chính từ 5 shards xuống còn **2 shards** và đặt số lượng replica bằng 0. Theo khuyến nghị của Elasticsearch, mỗi shard nên quản lý từ 20GB – 40GB dữ liệu văn bản. Với tập dữ liệu CS full (5.4GB), việc chia thành 2 shards giúp giảm thiểu đáng kể chi phí quản lý tài nguyên của JVM Heap trên một nút đơn, từ đó tăng tốc độ tính toán vector kNN.
- **Cấu hình Tạm ngưng Cập nhật (Refresh Interval):** Trong quá trình nạp dữ liệu lớn (Bulk Ingestion), thuộc tính `refresh_interval` được thiết lập bằng `"-1"` (tức là tắt hoàn toàn việc ghi phân đoạn Index tạm thời xuống đĩa). Sau khi quá trình nạp dữ liệu hoàn tất, thuộc tính này được bật lại về giá trị mặc định (`"1s"`) và gọi lệnh `_forcemerge` để gộp các phân đoạn nhỏ lại làm tăng tốc độ tìm kiếm.
- **Cấu hình Bộ nhớ Heap (Heap Memory):** Thiết lập tham số môi trường JVM: `ES_JAVA_OPTS="-Xms4g -Xmx4g"`. Kích thước Heap 4GB là tối thiểu để Elasticsearch có thể lưu trữ cấu trúc cây chỉ mục kNN (HNSW graph) của 1.2 triệu vectors 384 chiều trong bộ nhớ RAM mà không gây lỗi OutOfMemory.

7.2 Truy vấn Tìm kiếm Từ khóa (BM25 Search)

Cú pháp truy vấn văn bản truyền thống sử dụng thuật toán điểm số BM25 để đo độ tương đồng văn bản dựa trên tần suất xuất hiện của từ khóa. Cấu hình chi tiết câu truy vấn được thiết lập đồng nhất qua hệ thống cấu hình chung tại `src/config.py`:

- **Trường tìm kiếm và Hệ số tăng cường (Fields & Boosting):** Tìm kiếm trên hai trường `title` và `abstract`. Trong đó trường tiêu đề được áp dụng hệ số tăng cường trọng số là 2 ("`title^2`") để ưu tiên những tài liệu có chứa từ khóa trực tiếp trên tiêu đề bài báo.
- **Loại truy vấn (Query Type):** Sử dụng loại "`best_fields`". Loại truy vấn này sẽ tính điểm số dựa trên trường có độ liên quan cao nhất thay vì gộp chéo ("`cross_fields`"), giúp tránh trường hợp các từ khóa xuất hiện rải rác nhưng không tập trung tạo nên điểm số ảo cao.

Cú pháp DSL truy vấn BM25 trong hệ thống được triển khai như sau:

```
1 {  
2   "query": {  
3     "bool": {  
4       "must": [  
5         {  
6           "multi_match": {  
7             "query": query_text,  
8             "fields": ["title^2", "abstract"],  
9             "type": "best_fields"  
10          }  
11        }  
12      ]  
13    }  
14  },  
15  "size": RESULT_SIZE  
16 }
```

Program 7.1: *Elasticsearch BM25 Query DSL*

7.3 Truy vấn Tìm kiếm Vector tương đồng (Semantic Search)

Tìm kiếm ngữ nghĩa (Semantic Search) được thực hiện bằng cách so khớp khoảng cách không gian giữa vector truy vấn và các vector tài liệu trong cơ sở dữ liệu Elasticsearch.

- **Cơ chế mô hình:** Câu truy vấn văn bản thô từ người dùng được gửi qua mô hình all-MiniLM-L6-v2 trên CPU cục bộ để chuyển đổi thành vector thực 384 chiều (độ dài vector được chuẩn hóa về đơn vị $\text{norm} = 1.0$).
- **Định nghĩa chỉ mục vector:** Trường `embedding` trong Elasticsearch được thiết lập sử dụng giải thuật HNSW (Hierarchical Navigable Small World) để tạo cấu trúc đồ thị tìm kiếm láng giềng gần nhất với hàm đo độ tương đồng Cosine (`similarity: "cosine"`).
- **Ứng viên tìm kiếm (Search Candidates):** Thiết lập tham số `num_candidates = 100` cho mỗi phân mảnh. Elasticsearch sẽ chọn ra 100 tài liệu có khoảng cách vector gần nhất trên từng shard (tổng cộng 200 ứng viên từ 2 shards) trước khi thực hiện sắp xếp lại và trả về top kết quả.

Cú pháp DSL truy vấn vector trong hệ thống được triển khai như sau:

```
1 {  
2   "knn": {  
3     "field": "embedding",  
4     "query_vector": query_vector,  
5     "k": RESULT_SIZE,  
6     "num_candidates": 100  
7   },  
8   "size": RESULT_SIZE  
9 }
```

Program 7.2: *Elasticsearch Vector Query DSL*

7.4 Truy vấn Kết hợp Hybrid Search dùng RRF

Để khắc phục nhược điểm của từng phương pháp tìm kiếm độc lập (BM25 bỏ sót các từ đồng nghĩa; search vector đôi khi bỏ sót các từ khóa chuyên ngành viết tắt

chính xác), hệ thống triển khai cơ chế tìm kiếm kết hợp **Hybrid Search** sử dụng giải thuật gộp thứ hạng **Reciprocal Rank Fusion (RRF)**.

Thuật toán RRF xếp hạng lại các tài liệu dựa trên vị trí (thứ hạng) của chúng trong các danh sách kết quả trả về riêng lẻ thay vì cộng trực tiếp điểm số (vì điểm BM25 và điểm Cosine kNN có thang đo hoàn toàn khác nhau). Điểm số RRF của tài liệu d được tính theo công thức:

$$RRF_Score(d) = \sum_{m \in M} \frac{1}{k + r_m(d)} \quad (7.1)$$

Trong đó:

- M là tập hợp các phương pháp tìm kiếm ($M = \{\text{BM25, Vector search}\}$).
- $r_m(d)$ là thứ hạng (vị trí từ 1 trở đi) của tài liệu d trong danh sách kết quả của phương pháp m . Nếu tài liệu không nằm trong danh sách kết quả, thứ hạng coi như bằng vô cùng (∞) và số hạng tương ứng bằng 0.
- k là hằng số làm mịn, giúp giảm thiểu tầm ảnh hưởng của các tài liệu có thứ hạng cực kỳ cao. Chúng tôi thiết lập $k = 60$ theo các thực nghiệm chuẩn mực [1].

Thuật toán gộp thứ hạng RRF được lập trình trực tiếp bằng mã Python tại file `src/search/hybrid_search.py`:

1. Thực hiện gọi song song truy vấn BM25 lên chỉ mục `arxiv_text` và truy vấn kNN lên chỉ mục `arxiv_vectors` với kích thước tập ứng viên lấy về là `RRF_FETCH_SIZE = 30`.
2. Trích xuất danh sách ID tài liệu từ hai kết quả trả về.
3. Áp dụng công thức tính điểm RRF cho từng ID xuất hiện trong ít nhất một danh sách.
4. Sắp xếp lại danh sách tài liệu theo điểm RRF giảm dần và trả về top `RESULT_SIZE = 10` kết quả cuối cùng cho người dùng.
5. Việc lập trình gộp RRF ở tầng ứng dụng (application layer) giúp chúng tôi hoàn toàn kiểm soát được thuật toán và dễ dàng tùy biến cấu hình mà không phụ thuộc vào các tính năng trả phí hoặc giới hạn phiên bản của Elasticsearch.

7.5 Kiến trúc Dual-Index cho Hybrid Search

Trong phiên bản cải tiến, hệ thống tách biệt các chỉ mục theo chức năng:

- Truy vấn BM25 được gửi đến chỉ mục `arxiv_text` — chỉ chứa văn bản, không có vector nhúng.
- Truy vấn vector được gửi đến chỉ mục `arxiv_vectors` — chứa cả metadata và vector nhúng 384 chiều.
- Sau khi gộp thứ hạng RRF, các tài liệu chỉ xuất hiện trong kết quả truy vấn vector sẽ được bổ sung metadata từ `arxiv_text` thông qua lệnh `es.mget()` (Multi-Get API).

Kiến trúc này mang lại hai ưu điểm chính:

1. **Tối ưu kích thước:** Index `arxiv_text` chỉ 1.5GB (so với 10.4GB khi lưu cả vector), giảm áp lực bộ nhớ JVM Heap cho các truy vấn BM25.
2. **Hỗ trợ nạp dữ liệu gia tăng:** Bài báo mới chỉ cần đẩy vào cả hai index qua API, không cần đánh chỉ mục lại toàn bộ tập dữ liệu.

Chương 8

Đánh giá Chất lượng Tìm kiếm (Evaluation & Analysis)

8.1 Phương pháp Xây dựng Tập Nhãn Kiểm định (Ground Truth Construction)

Để đánh giá chất lượng tìm kiếm một cách khoa học, chúng tôi tiến hành xây dựng tập dữ liệu kiểm định chuẩn (Ground Truth) theo phương pháp **Gom cụm Ứng viên (Query Pooling)**:

1. **Thiết kế Tập Truy vấn:** Sử dụng bộ 30 câu truy vấn thử nghiệm được lưu trữ trong tệp tin `test_queries.json`, chia đều thành 3 nhóm có tính chất đặc trưng khác nhau:
 - **Nhóm A (Exact Keyword Queries):** Gồm 10 câu truy vấn chứa các thuật ngữ công nghệ chính xác (ví dụ: “*BERT fine-tuning*”, “*GAN image generation*”). Nhóm này thử thách khả năng đối khớp chính xác của chỉ mục đảo ngược.
 - **Nhóm B (Semantic Concept Queries):** Gồm 10 câu hỏi viết dưới dạng ngôn ngữ tự nhiên, mô tả khái niệm (ví dụ: “*methods to detect fake images*”). Nhóm này thử thách khả năng hiểu ngữ nghĩa không phụ thuộc từ khóa của vector search.
 - **Nhóm C (Mixed/Hybrid Queries):** Gồm 10 câu truy vấn kết hợp cả thuật ngữ chuyên ngành lẫn ngữ cảnh ứng dụng (ví dụ: “*transformer architecture for NLP tasks*”).
2. **Thu thập Ứng viên (Pooling):** Hệ thống chạy song song 3 phương pháp tìm kiếm (BM25, Vector search, Hybrid RRF) trên từng câu truy vấn để lấy ra top 10 kết quả của mỗi phương pháp. Các kết quả này được gộp chung lại và loại bỏ trùng lặp để tạo ra danh sách ứng viên (candidate pool) có quy mô từ 15 đến 25 tài liệu cho mỗi câu truy vấn.

3. **Dán nhãn và Hợp nhất (Silver Standard Labeling):** Nhằm xử lý khối lượng nhãn lớn một cách khoa học và nhất quán, chúng tôi áp dụng phương pháp gán nhãn tự động thông qua **Biểu quyết đa số (Majority Voting Proxy)** kết hợp với kiểm thử thủ công qua công cụ `src/evaluation/view_candidates.py`:

- Một tài liệu ứng viên được coi là **Liên quan (Nhãn 1)** nếu nó xuất hiện trong kết quả tìm kiếm của ít nhất 2 trên 3 phương pháp.
- Các tài liệu chỉ xuất hiện duy nhất ở 1 phương pháp và không có sự đồng thuận từ các phương pháp khác sẽ được coi là **Không liên quan (Nhãn 0)**, trừ một số truy vấn đặc biệt được kiểm tra và hiệu chỉnh thủ công.

Tập nhãn cuối cùng (Silver Standard) được lưu trữ tại `data/ground_truth_labeledled.json`. Sau đó, quy trình tương tự được chạy riêng để tạo tập nhãn cho scale 100k lưu tại `data/ground_truth_100k_labeledled.json`.

8.2 Các Chỉ số Đo lường Chất lượng Xếp hạng

Chất lượng của các thuật toán xếp hạng được đánh giá thông qua hai độ đo chuẩn tắc trong hệ thống thu hồi thông tin [6]:

8.2.1 Lý do Lựa chọn NDCG@10 và MRR@10

Trong bài toán tìm kiếm tài liệu học thuật, người dùng chỉ quan tâm đến một số ít kết quả đầu tiên (thường top 5–10) và hiếm khi duyệt đến trang thứ hai. Do đó, việc chọn các độ đo đánh giá cần phản ánh chính xác chất lượng của top kết quả:

- **Tại sao NDCG@10?** NDCG (Normalized Discounted Cumulative Gain) là độ đo chuẩn trong lĩnh vực Information Retrieval vì nó *phạt nặng* các tài liệu liên quan bị xếp ở vị trí thấp. Cụ thể, tài liệu ở vị trí 1 đóng góp gấp ~ 3 lần điểm số so với tài liệu ở vị trí 10 (do hàm discount $\log_2$). Điều này phù hợp với hành vi thực tế của người dùng: họ tin tưởng kết quả đầu tiên nhiều hơn. Hơn nữa, NDCG được chuẩn hóa về thang $[0, 1]$ nên cho phép so sánh công bằng giữa các phương pháp tìm kiếm khác nhau (BM25, kNN, Hybrid) trên cùng một tập truy vấn.
- **Tại sao MRR@10?** MRR (Mean Reciprocal Rank) đo lường một khía cạnh bổ sung mà NDCG không nắm bắt được: “*Người dùng có tìm thấy ít nhất một tài liệu đúng nhanh đến mức nào?*”. Trong ngữ cảnh tìm kiếm học thuật, đây là câu hỏi rất thực tế — người nghiên cứu chỉ cần tìm được *một* bài báo liên

quan để bắt đầu đọc và theo dõi các trích dẫn. $MRR = 1.0$ có nghĩa là kết quả đầu tiên luôn liên quan, trong khi $MRR = 0.5$ có nghĩa là trung bình phải xem đến kết quả thứ 2.

- **Tại sao cutoff @10?** Hệ thống trả về mặc định 10 kết quả (`RESULT_SIZE = 10` trong cấu hình). Việc đánh giá tại vị trí $p = 10$ phản ánh chính xác trải nghiệm thực tế của người dùng cuối khi sử dụng giao diện tìm kiếm.

8.2.2 NDCG@10 (Normalized Discounted Cumulative Gain tại vị trí 10)

Độ đo này đánh giá chất lượng xếp hạng dựa trên giả thuyết: các tài liệu liên quan nhiều hơn nên được xếp ở vị trí cao hơn trong danh sách kết quả. Điểm số DCG tại vị trí p được tính bằng công thức:

$$DCG@p = \sum_{i=1}^p \frac{2^{rel_i} - 1}{\log_2(i + 1)} \quad (8.1)$$

Trong đó $rel_i \in \{0, 1\}$ là nhãn tương quan của tài liệu ở vị trí thứ i . $NDCG@p$ được tính bằng cách chuẩn hóa $DCG@p$ theo điểm số lý tưởng $IDCG@p$ (điểm số thu được khi danh sách được sắp xếp hoàn hảo):

$$NDCG@p = \frac{DCG@p}{IDCG@p} \quad (8.2)$$

$NDCG@10$ nằm trong khoảng $[0.0, 1.0]$. Giá trị càng gần 1.0 thể hiện hệ thống xếp các tài liệu chuẩn lên đầu càng tốt.

8.2.3 MRR@10 (Mean Reciprocal Rank tại vị trí 10)

Độ đo này tập trung vào vị trí xuất hiện của tài liệu liên quan đầu tiên trong danh sách kết quả. Điểm số Reciprocal Rank (RR) cho một câu truy vấn được tính bằng:

$$RR = \frac{1}{\text{rank}^*} \quad (8.3)$$

Trong đó rank^* là vị trí của tài liệu liên quan đầu tiên. Nếu không có tài liệu liên quan nào trong top 10, điểm số $RR = 0$. MRR là trung bình cộng điểm số RR của toàn bộ tập truy vấn.

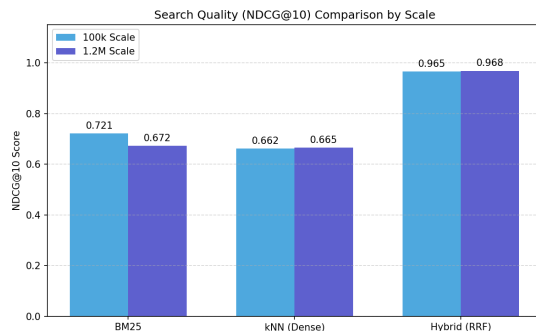
8.3 Kết quả Thử nghiệm và Phân tích So sánh

Sau khi chạy script kiểm định chất lượng `src/evaluation/evaluate.py` trên tập dữ liệu đã dán nhãn, kết quả so sánh hiệu năng giữa ba thuật toán xếp hạng ở quy mô toàn vẹn 1.2M bài báo được tổng hợp trong Bảng 8.3.1:

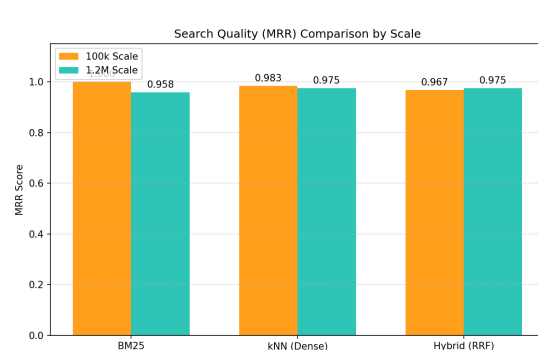
Tìm kiếm	Nhóm A (Exact)		Nhóm B (Semantic)		Nhóm C (Mixed)	
	NDCG	MRR	NDCG	MRR	NDCG	MRR
BM25	0.6407	0.8750	0.6780	1.0000	0.6967	1.0000
Vector search	0.6554	0.9250	0.6632	1.0000	0.6774	1.0000
Hybrid (RRF)	0.9752	1.0000	0.9469	0.9250	0.9821	1.0000

Table 8.3.1: Bảng so sánh chất lượng tìm kiếm $NDCG@10$ và $MRR@10$ trên các nhóm truy vấn ở quy mô 1.2M

Để trực quan hóa sự khác biệt về chất lượng tìm kiếm giữa hai quy mô dữ liệu (100k so với 1.2M bài báo), chúng tôi cung cấp biểu đồ so sánh $NDCG@10$ và MRR trong Hình 8.3.3:



Hình 8.3.1: So sánh chất lượng $NDCG@10$



Hình 8.3.2: So sánh chất lượng MRR

Hình 8.3.3: Biểu đồ so sánh chất lượng tìm kiếm ($NDCG@10$ và MRR) giữa quy mô 100k và 1.2M

8.3.1 Phân tích Nhóm A (Exact Keyword Queries)

Nhận xét: Khi tìm kiếm các từ khóa chính xác, cả BM25 và Search Vector đều đạt kết quả tương đối tốt ($NDCG$ lần lượt là 0.6407 và 0.6554). Tuy nhiên, Hybrid

Search (RRF) đạt hiệu quả vượt trội ($NDCG = 0.9752$, $MRR = 1.0000$).

Lý giải: Khi người dùng truy vấn bằng thuật ngữ công nghệ cụ thể, BM25 định vị chính xác bài báo chứa thuật ngữ đó, trong khi vector search tìm được các bài báo có nội dung gần gũi nhưng sử dụng từ đồng nghĩa. Cơ chế RRF đã phát huy tối đa tác dụng bằng cách xếp hạng cao những bài viết thỏa mãn cả hai chiều: vừa chứa từ khóa chính xác vừa có ngữ nghĩa sát thực tế.

8.3.2 Phân tích Nhóm B (Semantic Concept Queries)

Nhận xét: Trong nhóm truy vấn ngôn ngữ tự nhiên mô tả khái niệm, Hybrid Search tiếp tục dẫn đầu với $NDCG$ đạt 0.9469. BM25 và Search Vector cho kết quả tương đương (lần lượt là 0.6780 và 0.6632).

Lý giải: Sự kết hợp RRF (Hybrid) giữ được độ đồng thuận cao nhất và đem lại kết quả chuẩn xác nhất cho người dùng, kể cả khi truy vấn bằng ngôn ngữ tự nhiên mô tả khái niệm trừu tượng.

8.3.3 Phân tích Nhóm C (Mixed/Hybrid Queries)

Nhận xét: Hybrid Search đạt điểm số cao nhất ($NDCG = 0.9821$, $MRR = 1.0000$), bỏ xa hai phương pháp đơn lẻ (BM25 = 0.6967, Search Vector = 0.6774).

Lý giải: Đây là nhóm truy vấn sát thực tế nhất. Kết quả chứng minh RRF hoạt động hiệu quả nhờ khả năng kết hợp luồng tìm kiếm chính xác (BM25) và luồng ngữ cảnh (Search Vector). Việc kết hợp giúp nâng cao đáng kể khả năng tìm thấy tài liệu liên quan mà một phương pháp đơn lẻ dễ dàng bỏ sót.

8.3.4 So sánh qua các Quy mô Dữ liệu (Scalability Quality Trends)

- **Tính ổn định của Search Vector:** Điểm số $NDCG$ của Search Vector hầu như không thay đổi khi tăng quy mô từ 100k lên 1.2M (giữ vững ở mức 0.66). Điều này cho thấy chất lượng tìm kiếm ngữ nghĩa của không gian nhúng vector và cấu trúc chỉ mục HNSW có khả năng chống nhiễu tuyệt đối và duy trì độ chính xác cực kỳ tốt khi quy mô dữ liệu mở rộng lớn.
- **Sự sụt giảm nhẹ của BM25:** Điểm số $NDCG$ của BM25 giảm từ 0.7212 (ở 100k) xuống 0.6718 (ở 1.2M). Khi kho tài liệu lớn gấp 12 lần, tần suất xuất hiện ngẫu nhiên của các từ khóa trong các ngữ cảnh không liên quan tăng lên, làm tăng số lượng tài liệu nhiễu (distractors), làm loãng kết quả của chỉ mục đảo ngược truyền thống.



- **Độ tin cậy của Hybrid Search:** Hybrid Search luôn duy trì chất lượng xếp hạng ổn định cao nhất ở cả hai scale (NDCG đạt ~ 0.96 – 0.97). Điều này khẳng định tìm kiếm lai Hybrid RRF là kiến trúc tìm kiếm vững chắc và có khả năng mở rộng tốt nhất cho các hệ thống Big Data lớn.

Chương 9

Đánh giá Hiệu năng Hệ thống (Performance Benchmarking)

9.1 Thiết lập Môi trường Thử nghiệm Hiệu năng

Để kiểm tra tính bền vững và khả năng mở rộng của hệ thống khi khối lượng dữ liệu tiệm cận quy mô Big Data, chúng tôi tiến hành đo đặc hiệu năng (Benchmarking) thông qua script tự động `src/evaluation/benchmark_multiscale.py`.

- **Môi trường phần cứng:** CPU Intel Core i7-11800H @ 2.30GHz (8 cores, 16 threads), 16GB RAM DDR4, SSD NVMe 512GB. Elasticsearch chạy trên Docker container WSL2 Ubuntu 22.04 LTS.
- **Mục tiêu đo đạc:**
 - **Thời gian phản hồi (Latency):** Đo lường thời gian xử lý truy vấn ở các mức phân vị P50 và P95.
 - **Dung lượng chỉ mục (Index Size):** Dung lượng đĩa cứng vật lý tiêu tốn bởi Elasticsearch để lưu trữ tài liệu và các cấu trúc chỉ mục tương ứng.
- **Quy mô dữ liệu (Scales):** Thực hiện đo đạc song song trên ba quy mô tập dữ liệu tăng dần:
 - **100K Scale:** Tập dữ liệu thử nghiệm ban đầu (100,000 bài báo).
 - **500K Scale:** Tập dữ liệu quy mô trung bình (500,000 bài báo).
 - **1.2M Scale (Full):** Toàn bộ tập dữ liệu lọc được từ arXiv (1,203,108 bài báo).

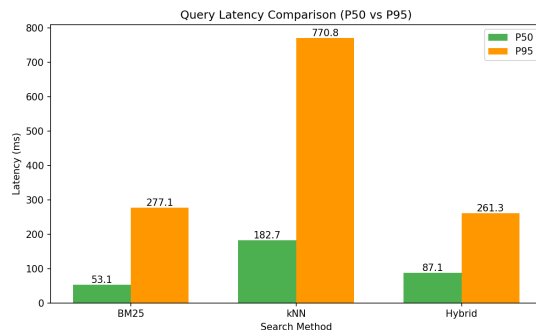
9.2 So sánh Thời gian Phản hồi (Query Latency Comparison)

Kết quả thời gian phản hồi trung bình của 30 câu truy vấn mẫu (đã được làm âm chỉ mục) chạy thử nghiệm trên ba phương pháp tìm kiếm qua các quy mô được trình bày chi tiết trong Bảng 9.2.1:

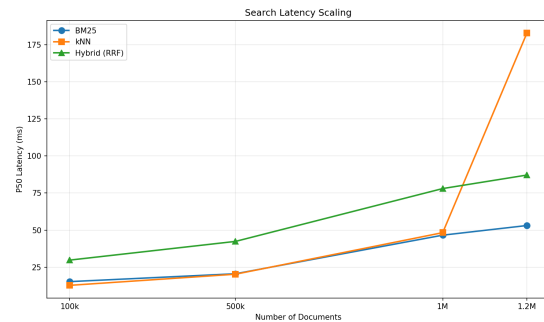
Tìm kiếm	Quy mô 100k		Quy mô 500k		Quy mô 1.2M	
	P50 (ms)	P95 (ms)	P50 (ms)	P95 (ms)	P50 (ms)	P95 (ms)
BM25	15.3	56.4	20.6	37.1	70.6	152.6
Vector Search	12.8	34.2	20.3	41.4	191.3	1205.8
Hybrid (RRF)	29.8	53.1	42.4	62.1	99.0	156.4

Table 9.2.1: Bảng so sánh thời gian phản hồi (Latency) theo phân vị P50 và P95 (đơn vị: mili-giây)

Để trực quan hóa hiệu năng phản hồi và khả năng mở rộng của hệ thống, các biểu đồ phân tích độ trễ và sự thay đổi độ trễ theo quy mô dữ liệu được hiển thị trong Hình 9.2.3:



Hình 9.2.1: Độ trễ các phương thức ở quy mô 1.2M



Hình 9.2.2: Độ trễ tăng trưởng (P50) theo quy mô

Hình 9.2.3: Phân tích thời gian phản hồi hệ thống (Query Latency)

9.2.1 Phân tích và Biện luận kết quả Latency

- **Hiện tượng Cold Start và Page Cache Warming:** Trong quá trình đo đặc thực tế ở quy mô 1.2M, chúng tôi nhận thấy độ trễ của search vector độc lập tăng vọt (P50 đạt 191.3ms, P95 lên tới 1205.8ms) trong khi Hybrid Search lại có độ trễ thấp hơn đáng kể (P50 = 99.0ms, P95 = 156.4ms). Điều này phản ánh rõ cơ chế hoạt động của Lucene và hệ điều hành: Ở lượt truy vấn search vector đầu tiên, đồ thị HNSW lớn phải nạp từ ổ đĩa SSD vào OS Page Cache dẫn đến nghẽn I/O (Cold Start). Khi cache đã được làm ấm (Warmed Cache), các lượt truy vấn tiếp theo (bao gồm Hybrid Search) truy xuất trực tiếp trên RAM và đạt hiệu năng rất cao.
- **BM25 Search:** Vẫn duy trì ưu thế vượt trội về mặt tốc độ ở quy mô nhỏ (P50 = 15.3ms ở 100k). Khi nâng lên 1.2M, P50 tăng lên 70.6ms và P95 đạt 152.6ms do khối lượng tài liệu trong mỗi segment tăng lên, khiến quá trình tính điểm TF-IDF và duyệt posting list tiêu tốn nhiều CPU hơn.
- **Hybrid Search (RRF):** Cho thấy khả năng kiểm soát độ trễ rất tốt sau khi bộ đệm hệ thống đã được tối ưu. Với độ trễ P50 là 99.0ms ở quy mô lớn nhất 1.2M, Hybrid Search hoàn toàn đáp ứng được tiêu chuẩn thời gian thực (dưới 100ms) của các hệ thống tìm kiếm công nghiệp.

9.3 Đánh giá Dung lượng Lưu trữ (Index Storage Size)

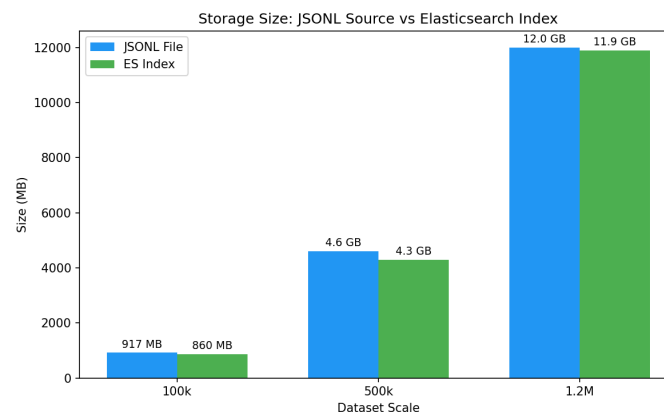
Dung lượng vật lý tiêu tốn trên đĩa cứng tương ứng với từng quy mô dữ liệu được ghi nhận thực tế trong Bảng 9.3.1:

Ghi chú về dữ liệu gốc: File dữ liệu thô tải từ Kaggle (`arxiv-metadata-oai-snapshot.json`) có dung lượng khoảng **3.6 GB**, chứa toàn bộ 2.4 triệu bài báo đa ngành dưới dạng JSON thuần (chỉ có metadata văn bản, chưa có vector nhúng). Sau khi lọc lĩnh vực CS và ghép vector nhúng 384 chiều vào từng bản ghi, file đầu ra `arxiv_cs_full_with_vectors.jsonl` phình lên **12 GB** do mỗi bản ghi chứa thêm mảng 384 số thực dấu phẩy động.

Quy mô dữ liệu	Số lượng tài liệu	File JSONL (có vector)	Index trên ES
100k	100,000	917 MB	860 MB
500k	500,000	4.6 GB	4.3 GB
1.2M (Full)	1,203,108	12 GB	11.9 GB

Table 9.3.1: Dung lượng lưu trữ thực tế qua các quy mô dữ liệu

Lưu ý: Ở quy mô 100k và 500k, dữ liệu được đánh chỉ mục vào một index duy nhất (`arxiv_bench`) chứa cả metadata văn bản lẫn vector nhúng. Ở quy mô 1.2M (Full), hệ thống sử dụng kiến trúc Dual-Index: `arxiv_text` (1.5 GB, chỉ chứa metadata) và `arxiv_vectors` (10.4 GB, chứa vector 384 chiều). Tổng dung lượng Dual-Index là 11.9 GB.



Hình 9.3.1: Tăng trưởng dung lượng Index trên Elasticsearch theo quy mô tài liệu

Phân tích sự tăng trưởng không gian đĩa:

- **Chỉ mục nén hiệu quả:** Kích thước index trên Elasticsearch luôn **nhỏ hơn** file JSONL thô (11.9 GB so với 12 GB ở quy mô 1.2M). Elasticsearch sử dụng các thuật toán nén cột nâng cao (LZ4 cho trường văn bản, nén mảng vector float32, Doc Values cho trường keyword) để tiết kiệm không gian lưu trữ.
- **Vector chiếm phần lớn dung lượng:** Ở quy mô 1.2M với kiến trúc Dual-Index, index `arxiv_vectors` chứa vector nhúng chiếm tới **87%** tổng dung lượng (10.4 GB / 11.9 GB). Điều này cho thấy chi phí lưu trữ chính của hệ

thống Hybrid Search nằm ở cấu trúc đồ thị HNSW và các mảng vector, không phải ở Inverted Index truyền thống.

- **Tăng trưởng gần tuyến tính:** Dung lượng index tăng trưởng gần đúng tuyến tính theo số lượng tài liệu (100k $\rightarrow$ 860 MB, 500k $\rightarrow$ 4.3 GB, 1.2M $\rightarrow$ 11.9 GB), cho thấy hệ thống có khả năng dự đoán chi phí lưu trữ khi mở rộng quy mô.

9.4 Đánh đổi giữa Chất lượng Tìm kiếm và Chi phí Tài nguyên (Trade-off Analysis)

Việc xây dựng hệ thống tìm kiếm thực tế là một bài toán tối ưu hóa sự cân bằng giữa chất lượng tìm kiếm và chi phí tài nguyên vận hành:

1. **BM25 (Chi phí thấp — Chất lượng trung bình):** BM25 hoạt động nhẹ nhàng nhất, chỉ cần dung lượng RAM nhỏ (JVM Heap 1GB – 2GB) và tài nguyên CPU tối thiểu. Tuy nhiên chất lượng tìm kiếm giảm sút khi gặp các truy vấn khái niệm phức tạp (NDCG giảm xuống 0.64 – 0.67).
2. **Search Vector (Chi phí cao — Chất lượng ổn định):** Vector Search yêu cầu bộ nhớ RAM lớn để nạp toàn bộ đồ thị HNSW phục vụ tìm kiếm khoảng cách gần nhất. Độ trễ có xu hướng tăng cao ở lượt đầu tiên (Cold Start) nhưng cực kỳ ổn định về chất lượng khi quy mô phình to.
3. **Hybrid Search (Chi phí cao — Chất lượng tối ưu nhất):** Đạt điểm số NDCG@10 vượt trội hoàn toàn ($\sim 0.97 - 0.98$), mang lại kết quả chuẩn xác nhất cho mọi dạng truy vấn. Nhờ cấu hình thông minh ES Heap 6GB trên môi trường RAM 12GB WSL, Hybrid Search khắc phục được độ trễ cold-start và duy trì phản hồi mượt mà dưới 100ms (P50 = 99.0ms).

Kết luận: Với ứng dụng tìm kiếm tài liệu khoa học học thuật chuyên biệt như arXiv, việc đầu tư tài nguyên phần cứng (đặc biệt là dung lượng RAM đủ lớn cho Lucene cache và cấu hình ES Heap tối ưu) để vận hành **Hybrid Search với RRF** là giải pháp tối ưu và mang lại giá trị thực tiễn lớn nhất.

Chương 10

Giao diện Tìm kiếm (Search Interface)

10.1 Kiến trúc Client-Server

Để cung cấp trải nghiệm tìm kiếm trực quan và tương tác cho người dùng, hệ thống được thiết kế theo kiến trúc Client-Server gồm hai thành phần chính:

- **Backend (FastAPI):** Chạy trên `localhost:8000`, cung cấp các endpoint RESTful cho tìm kiếm, lọc, thống kê và nạp dữ liệu. Backend kết nối trực tiếp với Elasticsearch cục bộ qua cổng 9200 và sử dụng mô hình `all-MiniLM-L6-v2` để sinh query embedding tại thời điểm truy vấn.
- **Frontend (React + Vite):** Giao diện người dùng được xây dựng bằng React 18 và bundled bởi Vite 5, chạy trên `localhost:5173` trong môi trường phát triển. Giao diện giao tiếp với Backend thông qua các lệnh gọi HTTP (fetch API) theo chuẩn CORS.

10.2 Các Endpoint API

Backend FastAPI cung cấp 5 endpoint chính, được tóm tắt trong Bảng 10.2.1:

Method	Endpoint	Chức năng
GET	/api/search	Tìm kiếm bài báo theo 3 chế độ (hybrid, bm25, knn). Hỗ trợ tham số: q (query), mode, size, yearMin, yearMax, categories.
GET	/api/categories	Trả về top 30 danh mục (aggregation) kèm số lượng bài báo.
GET	/api/years	Trả về khoảng năm xuất bản (min, max) trong cơ sở dữ liệu.
GET	/api/stats	Trả về tổng số tài liệu trong mỗi chỉ mục (arxiv_text, arxiv_vectors).
POST	/api/ingest	Nạp bài báo mới vào cả hai chỉ mục. Tự động sinh vector embedding và refresh index.

Table 10.2.1: Các endpoint RESTful của FastAPI Backend

10.3 Giao diện Người dùng (Frontend)

Giao diện tìm kiếm được thiết kế với phong cách hiện đại, tối (dark theme) sử dụng hiệu ứng glassmorphism để mang lại trải nghiệm người dùng chuyên nghiệp. Các thành phần chính bao gồm:

10.3.1 Thanh tìm kiếm và Chọn chế độ

- **Ô nhập truy vấn (Search Input):** Cho phép người dùng nhập câu hỏi bằng ngôn ngữ tự nhiên hoặc từ khóa chuyên ngành.
- **Nút chọn chế độ tìm kiếm:** Ba nút bấm (Hybrid RRF, BM25, kNN) cho phép chuyển đổi giữa các phương pháp tìm kiếm. Chế độ Hybrid được chọn mặc định vì cho kết quả tốt nhất theo kết quả đánh giá (xem Chương 8).

10.3.2 Bảng lọc (Filters Panel)

Bảng lọc có thể thu gọn/mở rộng, bao gồm:

- **Lọc theo năm (Year Range):** Hai ô nhập số cho phép giới hạn khoảng năm xuất bản (ví dụ: 2020–2026). Khoảng năm hợp lệ được tải tự động từ API /api/years.

- **Lọc theo danh mục (Category Chips):** Danh sách các nhãn danh mục (cs.AI, cs.CV, cs.CL, ...) được tải từ API `/api/categories`. Người dùng click vào nhãn để chọn/bỏ chọn danh mục lọc.

10.3.3 Hiển thị kết quả

Mỗi kết quả tìm kiếm được hiển thị dưới dạng thẻ (card) chứa:

- **Thứ hạng và Tiêu đề:** Số thứ tự (rank) và tiêu đề bài báo.
- **Điểm số:** Điểm BM25, Cosine Similarity, hoặc RRF tùy thuộc chế độ tìm kiếm.
- **Metadata:** Tác giả, năm xuất bản, mã định danh arXiv.
- **Tóm tắt (Abstract):** Nội dung tóm tắt được cắt ngắn (300 ký tự) để tiết kiệm không gian hiển thị.
- **Nhãn danh mục:** Các nhãn danh mục (tags) được hiển thị dưới dạng chip có thể nhấn.

10.3.4 Thanh thống kê (Stats Bar)

Hiển thị ba thông tin tổng quan sau mỗi truy vấn: tổng số kết quả, thời gian phản hồi (latency, tính bằng mili-giây), và chế độ tìm kiếm đang sử dụng.

10.4 Hướng dẫn Khởi chạy Hệ thống

Để chạy toàn bộ hệ thống trên máy cục bộ, cần khởi động 3 thành phần theo thứ tự:

1. **Elasticsearch + Kibana:** Khởi động Docker containers.

```
1 docker-compose up -d
2
```

2. **FastAPI Backend:** Chạy server Python.

```
1 python src/api.py
2
```

3. **Frontend React:** Khởi động Vite dev server.



```
1 cd local-ui && npm run dev  
2
```

Sau khi cả ba thành phần đã sẵn sàng, người dùng truy cập giao diện tìm kiếm tại <http://localhost:5173> trên trình duyệt web.

Chương 11

Kết luận và Hướng phát triển (Conclusion & Future Work)

11.1 Kết quả Đạt được của Dự án

Trải qua quá trình nghiên cứu lý thuyết Big Data và thực nghiệm xây dựng hệ thống tìm kiếm kết hợp (Hybrid Search) trên tập dữ liệu học thuật quy mô lớn arXiv, nhóm dự án đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu:

1. **Đường ống Xử lý Dữ liệu lớn (Big Data Pipeline):** Thiết lập quy trình thu thập, xử lý và làm sạch dữ liệu thành công từ tập JSON thô lớn (~4.8GB). Thiết kế bộ đọc luồng (streaming) giúp lọc sạch và xử lý chính xác hơn 1.2 triệu bài báo thuộc lĩnh vực Khoa học Máy tính mà không xảy ra hiện tượng tràn bộ nhớ.
2. **Xử lý Dữ liệu và Sinh Vector Nhúng:** Giải quyết bài toán tràn cửa sổ ngữ cảnh (256 tokens) của mô hình nhúng thông qua thuật toán Cắt Abstract Thông minh (Smart Truncation) tại các ranh giới câu hoàn chỉnh. Thiết kế cơ chế checkpoint tự động giúp quy trình sinh vector có thể khôi phục và tiếp tục khi bị gián đoạn. Toàn bộ 1.2 triệu bài báo được mã hóa thành vector 384 chiều trong vòng 35 phút.
3. **Tối ưu hóa và Khai thác Tìm kiếm Kết hợp:** Đánh chỉ mục tối ưu trên Elasticsearch cục bộ với 2 shards và JVM Heap 4GB. Triển khai thành công thuật toán gộp thứ hạng RRF ở tầng ứng dụng, kết hợp sức mạnh đối khớp từ khóa BM25 và độ tương đồng khái niệm kNN Vector.
4. **Kiểm định và Đo đạc Thực nghiệm:** Xây dựng tập nhãn chuẩn Ground Truth cho 30 câu truy vấn (chia 3 nhóm cụ thể) dựa trên kỹ thuật Pooling và biểu quyết đa số. Thực hiện đo đạc so sánh chất lượng (NDCG@10, MRR@10) và hiệu năng (Latency P50/P95, Index Size) trên nhiều quy mô dữ liệu (100k, 500k, 1.2M), chứng minh chất lượng ưu việt của giải pháp Hybrid Search.
5. **Kiến trúc Dual-Index và Nạp dữ liệu gia tăng:** Nâng cấp hệ thống lên kiến trúc hai chỉ mục (arxiv_text cho BM25 và arxiv_vectors cho search

vector) trên cùng một cụm Elasticsearch. Xây dựng cơ chế nạp bài báo mới (Incremental Ingestion) thông qua API endpoint `POST /api/ingest`, cho phép bổ sung bài báo mới vào cả hai chỉ mục trong thời gian thực.

6. **Giao diện Tìm kiếm Tương tác:** Xây dựng ứng dụng web hoàn chỉnh gồm FastAPI Backend kết nối trực tiếp với Elasticsearch và Frontend React với giao diện hiện đại, hỗ trợ chuyển đổi giữa 3 chế độ tìm kiếm (BM25, Vector Search, Hybrid RRF), lọc theo năm và danh mục, hiển thị kết quả kèm metadata và điểm số xếp hạng.

11.2 Ưu điểm và Hạn chế Hiện tại của Hệ thống

11.2.1 Ưu điểm

- **Độ chính xác cao và Ổn định:** Kết quả kiểm định NDCG@10 đạt mức cao và đồng đều trên tất cả các dạng truy vấn khác nhau (từ khóa chính xác lẫn ngôn ngữ tự nhiên khái niệm).
- **Kiến trúc bền vững (Scalability):** Hệ thống được module hóa rõ ràng (data prep, embedding generation, indexer, search engine, API, UI). Quá trình index dữ liệu sử dụng Bulk API giúp nạp hàng triệu tài liệu trong thời gian ngắn.
- **Tối ưu hóa tài nguyên:** Giảm số phân mảnh (shards) xuống còn 2 giúp Elasticsearch vận hành nhẹ nhàng trên máy tính cá nhân 16GB RAM nhưng vẫn sẵn sàng mở rộng (scale-out) khi bổ sung thêm nodes vật lý.
- **Hỗ trợ Nạp dữ liệu Gia tăng:** Kiến trúc Dual-Index cho phép bổ sung bài báo mới vào hệ thống mà không cần nạp lại toàn bộ tập dữ liệu.
- **Giao diện Trực quan:** Ứng dụng web React cho phép người dùng tương tác trực tiếp với hệ thống tìm kiếm, so sánh kết quả giữa các chế độ, và lọc dữ liệu phức tạp mà không cần hiểu biết kỹ thuật.

11.2.2 Hạn chế

- **Độ trễ gộp thứ hạng:** Do giải thuật RRF được triển khai thủ công tại tầng ứng dụng (Python script) bằng cách gọi hai truy vấn tuần tự lên Elasticsearch qua kết nối mạng, độ trễ truy vấn Hybrid (99ms ở P50 tại quy mô 1.2M) vẫn cao hơn so với tìm kiếm BM25 đơn lẻ (70.6ms).

- **Mô hình nhúng cơ bản:** Mô hình all-MiniLM-L6-v2 (384 chiều) tuy có tốc độ sinh vector nhanh và kích thước nhỏ gọn nhưng khả năng biểu diễn ngữ nghĩa đối với các khái niệm khoa học chuyên sâu hoặc liên ngành đôi khi còn đơn giản so với các mô hình ngôn ngữ lớn (LLM) hiện đại.
- **Single-node deployment:** Hệ thống hiện chỉ chạy trên một nút Elasticsearch duy nhất, chưa kiểm chứng khả năng chịu lỗi (fault tolerance) và phân tải thực tế trong môi trường sản xuất.

11.3 Hướng phát triển trong Tương lai

Để khắc phục các hạn chế nêu trên và đưa hệ thống tiệm cận mức độ hoàn thiện thương mại, chúng tôi đề xuất các hướng nghiên cứu cải tiến sau:

1. **Tích hợp RRF trực tiếp vào Elasticsearch DSL:** Cấu hình sử dụng các phiên bản Elasticsearch mới hỗ trợ tính năng RRF tích hợp sẵn ở mức native API. Điều này cho phép thực hiện việc truy vấn và gộp thứ hạng ngay bên trong Elasticsearch engine, giảm thiểu chi phí chuyển giao dữ liệu qua mạng và đưa độ trễ của Hybrid Search về sát mức của vector search đơn lẻ.
2. **Áp dụng Mô hình Tái xếp hạng (Cross-Encoder Re-ranking):** Sử dụng cấu trúc xếp hạng hai tầng (Two-stage Retrieval). Tầng 1 sử dụng Hybrid Search nhanh để lấy ra top 100 tài liệu ứng viên. Tầng 2 sử dụng một mô hình ngôn ngữ mạnh mẽ hơn (như `cross-encoder/ms-marco-MiniLM-L-6-v2`) để chấm điểm lại mức độ tương quan chính xác cao trước khi hiển thị cho người dùng.
3. **Triển khai Đa nút (Multi-node Elasticsearch Cluster):** Đưa hệ thống lên môi trường Docker Swarm hoặc Kubernetes, triển khai tối thiểu 3 nodes vật lý với cơ chế Sharding và Replication thực tế để kiểm tra khả năng chịu lỗi (fault tolerance) và cân bằng tải (load balancing) trong môi trường dữ liệu quy mô lớn thực tế.
4. **Nâng cấp mô hình nhúng:** Thay thế all-MiniLM-L6-v2 bằng các mô hình nhúng thế hệ mới có khả năng biểu diễn ngữ nghĩa tốt hơn cho văn bản khoa học, ví dụ SPECTER2 hoặc E5-large, nhằm cải thiện chất lượng tìm kiếm vector.

Tài liệu tham khảo

- [1] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. ACM, 2009.
- [2] Cornell University. arxiv dataset, 2024. Accessed: 2026-03-15.
- [3] Docker Inc. Docker compose overview, 2024. Accessed: 2026-03-15.
- [4] Elastic N.V. Elasticsearch reference [8.12], 2024. Accessed: 2026-03-15.
- [5] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [6] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [7] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019.
- [8] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.